

Intelligent Requests Orchestration for Microservice Management Based on Blockchain in Software Defined Networking: a Security Guarantee

1st Yasheng Zhang

2nd Chengcheng Li

The 54th Research Institute of CETC

Shijiazhuang 050081, China

zys163@163.com, lengcangche@bupt.cn

3rd Ning Chen*

4th Peiying Zhang

College of Computer Science and Technology

China University of Petroleum (East China)

Qingdao 266580, China

nchen@s.upc.edu.cn, zhangpeiying@upc.edu.cn

Abstract—Through the programmable control of the network, the distributed and discrete service equipment is managed uniformly, and the software-defined network (SDN) effectively improves the overall management and control capabilities of the centralized management model. Each service node in SDN is connected by the service function chain (SFC) and completes network services in a specified order. However, there are still the following challenges in service request orchestration of SDN: (1) The tightly coupled characteristic of SFC bring management problems, *e.g.*, deployment difficulties, service collisions, and poor scalability, *etc.*, to frequent user service requests. (2) The complicated technologies in SDN, *e.g.*, interference of wireless communication, cause potential safety hazards between service nodes, which may cause devastating effects on the controller. Correspondingly, we propose the following solutions: (1) Meaningfully transform the service request orchestration in SDN into the Microservice Management problem, and further combine Artificial Intelligence (AI) technology to provide a flexible, autonomous, and scalable intelligent service orchestration model. (2) Combine the Diffie-Hellman algorithm to establish a shared key for the communication node, and combine the characteristics of the distributed ledger in the consortium blockchain, *e.g.*, immutable, privacy, credibility, and high security, *etc.*, for identity authentication to further improve the security of the shared key. Eventually, we have proved through theory and practice that the proposed algorithm can provide efficient intelligent request orchestration extremely safely in SDN.

Index Terms—Software Defined Networking (SDN), Service Function Chain (SFC), Microservice Management, Blockchain, Request Orchestration, Artificial Intelligence (AI)

I. INTRODUCTION

In recent years, as emerging network technology, Software Defined Networking (SDN) has attracted widespread attention in the industry [1], [2]. It separates the control plane from the data forwarding plane and improves the programmability control of the network [3]. Specifically, it uniformly manages distributed and discrete hardware and software devices, *e.g.*, Internet of Things (IoT) devices, Servers, Load Balance (LB), Intrusion Detection System (IDS), Intrusion Prevention System (IPS), *etc.* Therefore, SDN can effectively improve the overall

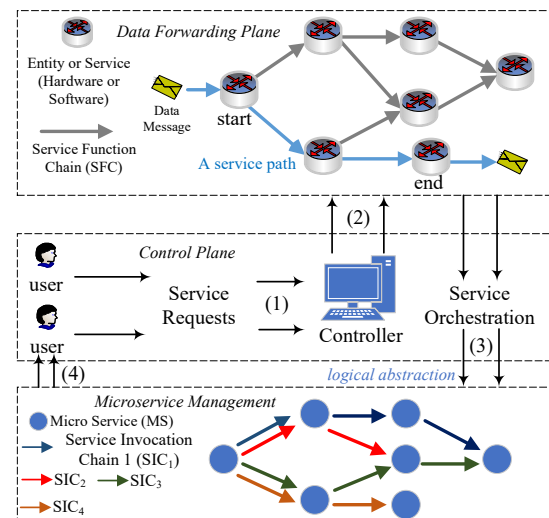


Fig. 1. Service requests orchestration for microservice management in SDN. The specific flow is: (1) Users send service requests; (2) The controller logically abstracts entities in the data forwarding layer plane; (3) Service orchestration through microservice management; (4) Respond to users requests.

management and control capabilities through this centralized management pattern [4].

As shown in Fig. 1, the realization of services in SDN requires data messages to run in the order specified by the business logic in the network entities. The communication link between different network entities is called the Service Function Chain (SFC), which constitutes the important service function. Specifically, SFC provides the function of defining an ordered list of network services, which are organized to create a service path [5]. When a data message enters the service path, it will flow through each service node (entity) according to the predetermined order of SFC, and then forward to the next node, finally encapsulate and forward at the last node to complete the specified network service [6]. We can find that SFC in SDN has the characteristic of tight coupling. When it is expanded or changed, it is necessary to make complex mod-

*Corresponding authors: Ning Chen

ifications to the topology, and frequent manual programming control is unrealistic. Therefore, in response to user requests, how to organize autonomous and extensible intelligent service request orchestration is a meaningful issue. Through logical abstraction, microservice management can provide flexible and intelligent service orchestration [7]. It resolves the problems of poor scalability, deployment difficulties, and bloated services in the monolithic system by decomposing related entities into mutually independent microservice (MS), and implementing specified services through logical calls between MS. It has a similar structure and functional characteristics as SFC, so inspired by it, to solve the above problems, we employ microservice management for SFC intelligent request orchestration.

Moreover, there are many types of SDN technologies. For example, SDN is implemented by different software (*e.g.* Dragonflow, Midonet, *etc.*), the control plane also has different processing technologies (*e.g.* centralized, distributed, active, hybrid, *etc.*). To provide reliable links, these technologies commonly utilize secure communication protocols for security control, *e.g.* OpenFlow. However, this approach still has potential safety hazards during the control gap. In addition, with the development of wireless technology, communications between different devices frequently interfere. Therefore, security in SDN is still an essential issue to be resolved [8], [9]. The emergence of Bitcoin based on a distributed, decentralized, and trusted system marked the beginning and success of blockchain technology. Blockchain, as a distributed ledger (system), has the characteristics of decentralization, autonomy, immutability, accuracy and credibility, anonymity and privacy, and high security, *etc.* [10] and is widely used in security management issues in non-secure environments [1], [8], [11]. In addition, according to different consensus algorithms, blockchains can also be divided into public chains, consortium chains, and private chains. According to the characteristics of microservices in SDN, the selected blockchain technology should be as lightweight as possible, low cost, fast processing speed, high security, and privacy. The consortium chain can suitably meet our expectations, so we employ it for the security deployment of microservice management to improve the security and privacy of user's request orchestration.

Inspired by the above, we have proposed a novel intelligent requests orchestration for microservice management based on blockchain. In particular, the contributions of this research are summarized as:

- Aiming at the deployment difficulties, service congestion, and poor scalability in service requests orchestration caused by frequent user requirements, we convert it into microservice management problems and combine AI technology to further provide flexible, autonomous, and scalable intelligent services orchestration.
- Aiming at the security and privacy issues of communication between microservices in SDN, we have combined the Diffie-Hellman algorithm to establish a symmetric shared key for the communication nodes, and combined with the consortium blockchain for identity access au-

thentication, so that the communication process cannot be tampered with and has high reliability and credibility.

- Through simulation to simulate the real environment for experiments, satisfactory results have been obtained in *long-term average revenue*, *acceptance rate*, and *long-term revenue-cost ratio*. Eventually, the effectiveness of the proposed algorithm is proved through theory and practice.

The rest of this paper is organized as follows: In Section II, the related work is briefly described; In Section III, the definition of the problem and the details of the proposed algorithm are elaborated; In Section IV, to prove the effectiveness, simulation experiments are carried out; In Section V, the work done is summarized.

II. RELATED WORK

A. Software Defined Networking

Compared with the scattered functions of the traditional IP network, SDN adopts centralized programmable management, which significantly improves the scalability and flexibility of the network. Works [2], [12] have re-examined and defined the SDN architecture. Due to it can realize network interconnection under the cooperation of the control layer, and dynamically configure and optimize network resources, it is widely used in various fields [4], such as Internet of Things (IoT) [13], Vehicle-to-Grid (V2G) [14], Internet of Vehicles (IoV) [3], *etc.*

B. Microservice Management

Microservice architecture is the mainstream method in the software industry. It performs vital business functions through highly cohesive, lightweight, and small services, which significantly improves the flexibility and scalability of the network [7]. Hannousse *et al.* [5] have further reviewed the mapping mechanism of the microservice system on the basis of Vale *et al.* [15], and focused on the security issues. Fu *et al.* [16] have applied microservices to Cloud-Edge Continuum to adaptively and efficiently deploy microservices, effectively reducing resource loss. In short, microservice management has been widely used due to its high flexibility.

C. Blockchain

Blockchain technology is widely used in various fields due to its privacy and transparency [11]. Singh *et al.* [8] have proposed to use the blockchain to create smart contracts to establish trust between both parties to the transaction and provide a secure communication framework for retail issues in the COVID-19 scenario. In the issue of SDN security, Aujla *et al.* [1] have proposed BlockSDN to deal with insecure factors such as man-in-the-middle attacks and applied it to smart cities. However, security issues in SDN are still the focus of continuous attention.

Aiming at the problems of deployment difficulties, service congestion, and poor service communication security that still exist in service request orchestration in SDN, we provide

TABLE I
THE NOTATIONS OF MICROSERVICE MANAGEMENT AND USER REQUESTS

Model	Notation	Definition
Microservice Management (M) in SDN	MS^M	The set of microservices, corresponding to nodes in SDN
	SIC^M	The set of service invocation chains, corresponding to SFC in SDN
	LOC^M	The location of microservices
	DEL^M	The delay of SIC^M in SDN
	RES^M	The resources of MS^M in SDN
	RES^M	The resources of SIC^M in SDN
User request (U)	MS^U	The set of required microservices
	SIC^U	The set of required service invocation chains
	LOC^U	The location of required microservices
	DEL^U	The delay of SIC^U
	RES^U	The resources of MS^U
	RES^U	The resources of SIC^U

flexible and intelligent orchestration strategies based on microservice management, and combine blockchain technology to improve communication reliability and security between microservices.

III. PROBLEM DEFINITION AND THE PROPOSED ALGORITHM

A. Notation

Table I records the definition notations used in this work. It should be noted that the resources of MS and SIC can have multiple forms, *e.g.*, CPU, bandwidth, *etc.* In this work, we use a unified form of representation. In fact, it can be easily extended to multiple resource problems. Moreover, lowercase letters are used to denote specific elements, *e.g.* sic_k^M indicates the k -th chain in M . If ms_i^M and ms_j^M are the start and end nodes of sic_k^M , respectively, they can also be expressed as $ms_i^M - ms_j^M$.

Different MS^U and SIC^U can coexist on the same MS^M and SIC^M respectively, which originates from frequent user requests. Therefore, the problem we studied can be summarized as: *how to consume the least resources to meet the user requests as many as possible under the premise of limited resources in M* . It can be known that it is an NP-hard problem [17]. Meanwhile, *how to ensure information such as service resources are not attacked and tampered with, i.e., access security* is also an important dispute.

B. Orchestration Constraints

In the request orchestration process, the corresponding resources are allocated to U . Part of the resources in M is occupied by different U , and this part of resources is not released until the service of U ends. In the orchestration process, the following constraints should be fulfilled:

(1) *Available resources of MS^M* : For ms_i^M , its resources should be sufficient to assign to ms_j^U in all user requests that can be satisfied, specifically denoted as:

$$REM^M(ms_i^M) - \sum_{j=1}^{|U|} \sum_{\forall (ms_i^M \rightarrow ms_j^U)} REM^U(ms_j^U) \geq 0, \quad (1)$$

where $|U|$ represents the number of user requests.

(2) *Available resources of SIC^M* : For sic_k^M , its resources should be sufficient to assign to sic_j^U in all user requests that can be satisfied, specifically denoted as:

$$RES^M(sic_k^M) - \sum_{j=1}^{|U|} \sum_{\forall (sic_k^M \rightarrow sic_j^U)} RES^U(sic_j^U) \geq 0. \quad (2)$$

(3) *Delay limits of SIC^M* : For sic_k^M , its delay should be greater than the delay of all SICs assigned to sic_j^U , specifically denoted as:

$$DEL^M(sic_k^M) \geq \max \{ DEL^U(sic_j^U) \} \quad (3)$$

$$s.t. \quad \forall (sic_k^M \rightarrow sic_j^U), 1 \leq j \leq |U|.$$

C. Diffie-Hellman to Establish the Symmetric Key

The Diffie-Hellman confidentiality protocol can enable two parties in communication to exchange keys securely in an insecure channel, so it is used to encrypt communication information. For example, for the requested microservices ms_1 and ms_2 , the shared key needs to be established for them to ensure the security of their information transmission. Specifically, the key exchange steps are:

(1) Randomly generate two prime numbers p and g , where g is the primitive root of p and satisfies the following equation:

$$g^i \bmod p = i, \quad s.t. \quad \forall i \in [1, p-1], \quad (4)$$

where $\bmod$ is the remainder operator.

(2) ms_1 randomly generates and saves its private key a , and ms_2 randomly generates and saves its private key b .

(3) ms_1 calculates its public key K_1 and sends it to ms_2 .

$$K_1 = g^a \bmod p. \quad (5)$$

(4) ms_2 calculates its public key K_2 and sends it to ms_1 .

$$K_2 = g^b \bmod p. \quad (6)$$

(5) ms_1 calculates and obtains the symmetric key K_{12} .

$$K_{12} = K_2^a \bmod p. \quad (7)$$

(6) ms_2 calculates and obtains the symmetric key K_{21} .

$$K_{21} = K_1^b \bmod p. \quad (8)$$

where $K_{12} = K_{21}$. It should be noted that a , b , and p must be very large, otherwise, the key can easily be exhaustively enumerated. g usually takes a small value, and an empirical value of 5 is set in this work. Take SIC_3 as an example, the shared symmetric key construction process between microservices is shown in Fig. 2. During the request orchestration process, the microservices must have the same symmetric key to communicate.

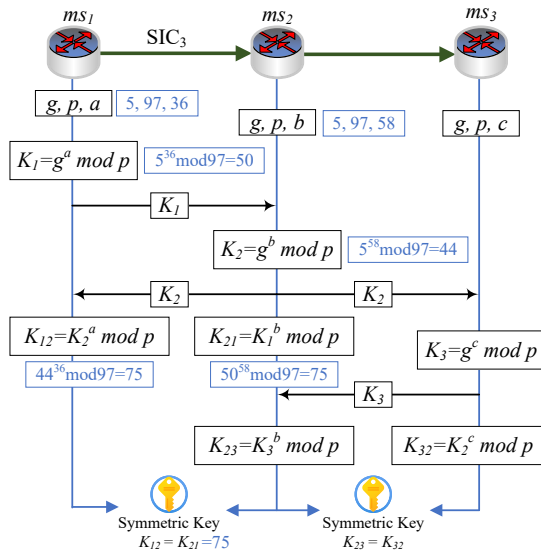


Fig. 2. The process of symmetric key construction between microservices, where the text in blue font is an actual example

D. Consortium Blockchain for Identity Authentication

However, the lack of identity authentication between the communicating parties makes it vulnerable to man-in-the-middle attacks. Therefore, we combine the blockchain technology to join the identity authentication mechanism to strengthen the security and privacy of the key exchange technology, and apply it to the SDN microservice management.

Correspondingly, each ms needs to register and record information in the consortium blockchain ledger, including prime number g , primitive root p , and its own public key K_* . After registration, key exchanges can be carried out. Meanwhile, both parties need to perform access authentication and create the symmetric share key. The records of key exchange will be recorded in the ledger and maintained by each node. Assuming that ms_1 needs to communicate with ms_2 , the key exchange process of introduced access authentication is illustrated in Fig. 3. The specific description is as follows:

(1) ms_1 generates the session (including the address of the sender, address of the receiver, prime number p , primitive root g , public key K_1 , private key signature a) and broadcasts it through the broadcast mechanism.

(2) ms_2 acts as a miner node to verify the sender's signature in the session is correct and credible, the session will be packaged into a block and added to the blockchain.

(3) Through the broadcast mechanism, ms_2 obtains the session information of ms_1 that wants to communicate with it. And query the information of ms_1 stored in the ledger at the registration stage. If the verification is inconsistent, the communication is rejected; after the verification is consistent, the *Diffie-Hellman* algorithm can be used to establish the Symmetric Key according to Section III-C.

(4) ms_2 creates the session, where the receiver is ms_1 , and the prime number and primitive root are obtained from the

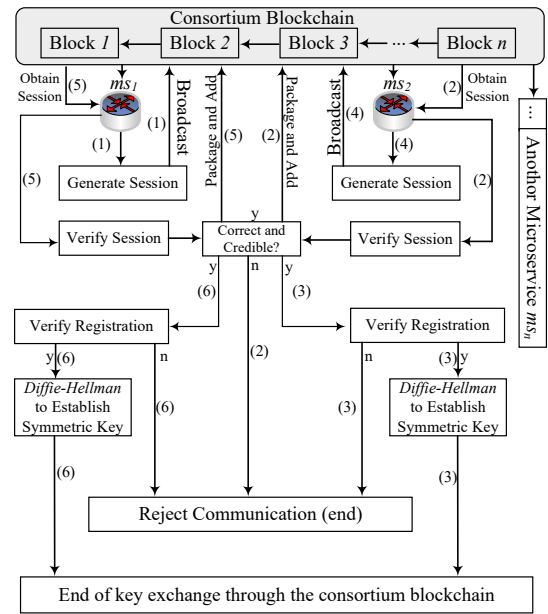


Fig. 3. The process of strengthening the security and privacy of key exchange technology through the consortium blockchain for identity authentication. It should be noted that the consortium blockchain is shared by all MS in SDN.

session information in the ms_1 block obtained in (3). Then broadcast the session, the process is similar to (1).

(5) ms_1 act as a miner node and perform the process similar to (2) and (3).

E. Request Orchestration

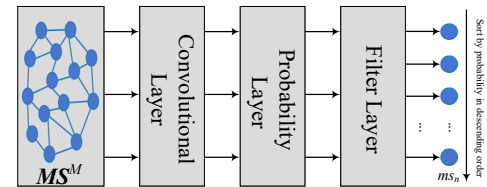


Fig. 4. The structure of orchestration network. Three layers: (1) Convolutional layer; (2) Probability layer; (3) Filter layer.

To provide autonomous and scalable smart service request orchestration, we have designed an orchestration network, which is mainly composed of a three-layer architecture and manifested in Fig. 4. Specifically, the convolutional layer is utilized to extract the local spatial features of MS^M to obtain resource vectors. The probability layer employs the *softmax* function to calculate the probability of MS^M being orchestrated according to the available resources. The filtering layer filters out the microservices that do not meet the constraints described in Section III-B and sorts them in descending order. Finally, breadth-first search is used to orchestrate SIC^M .

In addition, when orchestration the SICs, we first need to use the proposed consortium blockchain technology to add an identity authentication mechanism for both parties to enhance the security and privacy of the key exchange technology. Second, the symmetric key is created for the end microservices

of the SIC through the Diffie-Hellman algorithm. If any of the process information in the middle fails to pass the verification, the communication will be rejected, i.e., this service request orchestration will fail. Although it will reduce the success rate of request orchestration to a certain extent, it fully guarantees the security and reliability of the orchestration.

F. Evaluation indicators

Specifically, the following indicators are utilized to evaluate the resource consumption:

(1) *Cos*: the cost of the additional resources consumed for service orchestration. Therefore, it is not only related to REM^U but also to RES^U . In addition, the longer the SIC^U is requested, i.e. the greater the number of hops, the higher the cost.

$$Cos(M \rightarrow U) = \sum_{ms^U \in MS^U} REM^U(ms^U) + \sum_{sic^U \in SIC^U} hops(sic^U) \times RES^U(sic^U). \quad (9)$$

(2) *Rev*: the resources that successfully response to user requests, i.e. revenue.

$$Rev(M \rightarrow U) = \sum_{ms^U \in MS^U} REM^U(ms^U) + \sum_{sic^U \in SIC^U} RES^U(sic^U). \quad (10)$$

Accordingly, the following indicators is employed to evaluate the performance of the algorithm we proposed:

(1) *Long-term Average Revenue*: the integral of revenue over time.

$$Rev_L = \lim_{T \rightarrow +\infty} \frac{\sum_{t=0}^T Rev_t(M \rightarrow U)}{T}. \quad (11)$$

(2) *Acceptance Ratio*: the percentage of successfully responded requests U_{acc} to all user requests U .

$$Acc = \lim_{T \rightarrow +\infty} \frac{\sum_{t=0}^T U_{acc_t}}{\sum_{t=0}^T U_t}. \quad (12)$$

(3) *Long-term Revenue-Cost Ratio*: The less resource loss, the better the algorithm performance, so the ratio of long-term revenue to long-term cost is used for evaluation.

$$Rev_L/Cos_L = \lim_{T \rightarrow +\infty} \frac{\sum_{t=0}^T Rev_t(M \rightarrow U)}{\sum_{t=0}^T Cos_t(M \rightarrow U)}. \quad (13)$$

IV. EXPERIMENT

In this section, we conduct simulation experiments to evaluate the effectiveness of the proposed algorithm.

A. Simulation Environment

100 microservices and 581 SFCs are randomly generated. Randomly generate 2000 user requests U , the first 1000 as the training set, and the remainder as the testing set. In addition, computing power is taken as an example to represent REM , and bandwidth is taken as an example to represent RES . The detailed configuration of the simulation environment is recorded in Table II.

TABLE II
THE DETAILED CONFIGURATION OF THE SIMULATION ENVIRONMENT

Model	Simulation Enviroment	Configuration
M	DEL^M	1ms - 60ms
	REM^M	20TFLOPS - 100TFLOPS
	RES^M	50Mbps - 100 Mbps
U	DEL^U	1ms-50ms
	REM^U	1TFLOPS - 20TFLOPS
	RES^U	1Mbps - 20Mbps

TABLE III
AVERAGE PERFORMANCE

	Rev_L	Acc	Rev_L/Cos_L
$1ms \leq DEL^U \leq 50ms$	1720.8301	0.811	0.3392
$1ms \leq DEL^U \leq 40ms$	1586.1305	0.779	0.3288
$1ms \leq DEL^U \leq 30ms$	1505.3366	0.714	0.3447
$1ms \leq DEL^U \leq 20ms$	1116.8953	0.579	0.3511

B. Performance

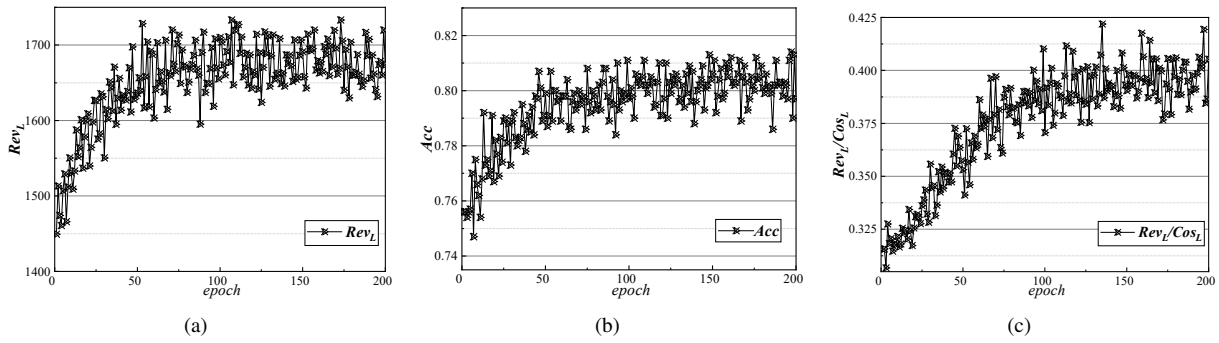
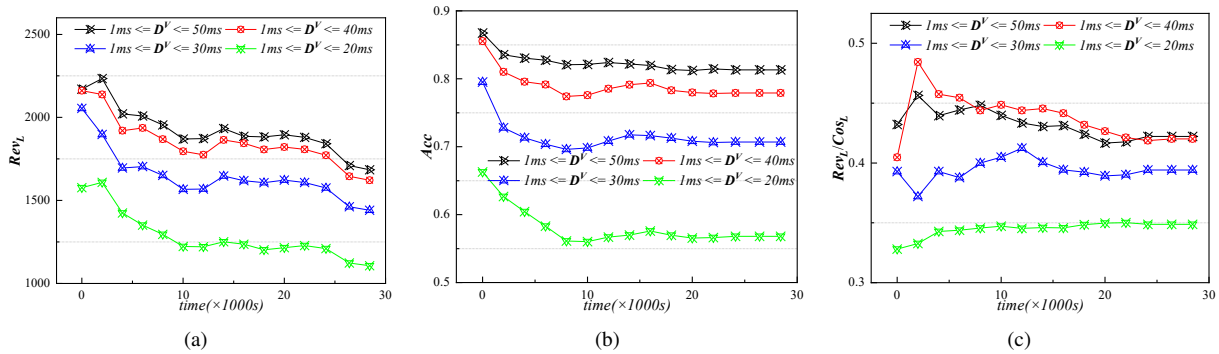
Through Fig. 5, we can find that the orchestration network converges quickly with the iterative process. And we observe the performance by limiting the upper limit of DEL^U to 20ms, 30ms, 40ms, and 50ms in sequence. As the upper limit is decreased, the number of user requests that can be successfully scheduled is reduced consequently, and the corresponding Rev_L and Acc are gradually reduced. Since Rev_L/Cos_L is also related to Cos_L , its fluctuation is slight. The results in Table III also confirm the correctness of our theoretical analysis. Moreover, as time goes by, allocable resources decrease, and the three indicators should show the same trend as above. The results are manifested in Fig. 6 also excellently prove this process. Therefore, our proposed orchestration network can converge quickly, and its training and testing phases are also effective and can orchestrate microservices to users safely and efficiently.

V. CONCLUSION

In this work, we propose a novel intelligent requests orchestration for microservice management based on blockchain in SDN. It provides a flexible and scalable intelligent service orchestration model for SDN through the combination of microservice management technology and AI technology. In addition, the Diffie-Hellman algorithm is combined to establish a shared symmetric key between microservices. And combined with the consortium blockchain to perform identity access authentication for communication microservices, so that the communication process has reliable security. In the end, the effectiveness of the proposed algorithm is proved from the perspective of theory and practice.

VI. ACKNOWLEDGEMENTS

This work is partially supported by the Shandong Provincial Natural Science Foundation, China under Grant ZR2020MF006, and partially supported by the Major Scientific and Technological Projects of CNPC under Grant ZD2019-183-006.

Fig. 5. The variation of Rev_L , Acc and Rev_L/Cos_L , with the iterative training process.Fig. 6. The variation of Rev_L , Acc and Rev_L/Cos_L , with the iterative training process.

REFERENCES

- [1] G. S. Aujla, M. Singh, A. Bose, N. Kumar, G. Han, and R. Buyya, "Blocksdn: Blockchain-as-a-service for software defined networking in smart city applications," *IEEE Network*, vol. 34, no. 2, pp. 83–91, 2020.
- [2] D. Kreutz, F. M. Ramos, P. E. Verissimo, C. E. Rothenberg, S. Azodolmolkly, and S. Uhlig, "Software-defined networking: A comprehensive survey," *Proceedings of the IEEE*, vol. 103, no. 1, pp. 14–76, 2014.
- [3] A. Gulati, G. S. Aujla, N. Kumar, S. Garg, and G. Kaddoum, "Software-defined content dissemination scheme for internet of healthcare vehicles in covid-like scenarios," *IEEE Internet of Things Magazine*, vol. 4, no. 3, pp. 34–40, 2021.
- [4] H. Cao, Y. Hu, S. Wu, J. Du, F. Tian, G. S. Aujla, and L. Yang, "Novisec: Novel virtual network mapping framework for secure software-defined networking," in *2020 IEEE Wireless Communications and Networking Conference Workshops (WCNCW)*. IEEE, 2020, pp. 1–6.
- [5] A. Hannousse and S. Yahiouche, "Securing microservices and microservice architectures: A systematic mapping study," *Computer Science Review*, vol. 41, p. 100415, 2021.
- [6] K. Kaur, S. Garg, G. S. Aujla, N. Kumar, J. J. Rodrigues, and M. Guizani, "Edge computing in the industrial internet of things environment: Software-defined-networks-based edge-cloud interplay," *IEEE communications magazine*, vol. 56, no. 2, pp. 44–51, 2018.
- [7] S. Garg, K. Kaur, G. Kaddoum, P. Garigipati, and G. S. Aujla, "Security in iot-driven mobile edge computing: new paradigms, challenges, and opportunities," *IEEE Network*, vol. 35, no. 5, pp. 298–305, 2021.
- [8] M. Singh, G. S. Aujla, R. S. Bali, S. Vashisht, A. Singh, and A. Jindal, "Blockchain-enabled secure communication for drone delivery: a case study in covid-like scenarios," in *Proceedings of the 2nd ACM MobiCom Workshop on Drone Assisted Wireless Communications for 5G and beyond*, 2020, pp. 25–30.
- [9] P. Zhang, F. Liu, N. Kumar, and G. S. Aujla, "Information classification strategy for blockchain-based secure sdn in iot scenario," in *IEEE INFOCOM 2020-IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*. IEEE, 2020, pp. 1081–1086.
- [10] W. Sun, L. Wang, P. Wang, and Y. Zhang, "Collaborative blockchain for space-air-ground integrated networks," *IEEE Wireless Communications*, vol. 27, no. 6, pp. 82–89, 2020.
- [11] D. Berdik, S. Otoum, N. Schmidt, D. Porter, and Y. Jararweh, "A survey on blockchain for information systems management and security," *Information Processing & Management*, vol. 58, no. 1, p. 102397, 2021.
- [12] W. Xia, Y. Wen, C. H. Foh, D. Niyato, and H. Xie, "A survey on software-defined networking," *IEEE Communications Surveys & Tutorials*, vol. 17, no. 1, pp. 27–51, 2014.
- [13] X. Ren, G. S. Aujla, A. Jindal, R. S. Bathth, and P. Zhang, "Adaptive recovery mechanism for sdn controllers in edge-cloud supported fintech applications," *IEEE Internet of Things Journal*, pp. 1–1, 2021.
- [14] A. Jindal, G. S. Aujla, and N. Kumar, "Survivor: A blockchain based edge-as-a-service framework for secure energy trading in sdn-enabled vehicle-to-grid environment," *Computer Networks*, vol. 153, pp. 36–48, 2019.
- [15] A. Pereira-Vale, G. Márquez, H. Astudillo, and E. B. Fernandez, "Security mechanisms used in microservices-based systems: A systematic mapping," in *2019 XLV Latin American Computing Conference (CLEI)*. IEEE, 2019, pp. 01–10.
- [16] K. Fu, W. Zhang, Q. Chen, D. Zeng, and M. Guo, "Adaptive resource efficient microservice deployment in cloud-edge continuum," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 8, pp. 1825–1840, 2022.
- [17] L. Gu, D. Zeng, J. Hu, B. Li, and H. Jin, "Layer aware microservice placement and request scheduling at the edge," in *IEEE INFOCOM 2021 - IEEE Conference on Computer Communications*, 2021, pp. 1–9.